INFORME FINAL

FutbolStats | Trabajo Práctico Integrador | Etapa 2

1. Diferencias respecto del diseño de la Etapa 1

El modelo de la Etapa 1 funcionó como guía para organizar las clases y responsabilidades. Durante la codificación se realizaron ajustes para adaptar el diseño a la estructura concreta del programa y facilitar su integración.

Priorizamos que el sistema funcione y cumpla las reglas de negocio, y al final actualizamos el diagrama para que coincida con el código: no se cambió ninguna regla para adaptarla al diagrama. El detalle completo está en docs/RESUMEN_CAMBIOS_UML_ETAPA2.md y en el diagrama actualizado (docs/diagramas_UML/Diagrama_Clases_Etapa2.drawio y su imagen PNG). El diagrama final de la Etapa 1 se conserva sin cambios para poder compararlos.

Aspecto	Cambio realizado
Interfaz	El diagrama de la Etapa 1 no tenía clases de interfaz. En la implementación se concretó una interfaz de consola con Menu y Consola, separando la entrada/salida de las reglas del dominio.
Controladores	Los tres controladores del diagrama final (ControladorTorneo, ControladorPartido y ControladorReportes) se implementaron tal como estaban previstos: el menú les pasa los pedidos y ellos los delegan en el modelo. Sus métodos reciben además los datos que necesitan (por ejemplo, la lista de suscriptores en ControladorPartido).
Persistencia	El diseño de la Etapa 1 no incluía clases de persistencia. Se agregaron RepositorioArchivo, que guarda los objetos y los recupera con carga tipada mediante Class, y la excepción PersistenciaException.
Objeto raíz de datos	La información que mantiene la aplicación se agrupa en Datos, que contiene las colecciones de torneos, clubes, jugadores, entrenadores, árbitros, estadios, suscriptores y sponsors. Esta instancia es la que se persiste en futbolstats.dat.
Torneo y fixture	Se agregó la clase Fixture (fechas y partidos, generados con el método del círculo) y el enum TipoTorneo. Torneo compone el Fixture, y el Fixture compone los Partido (antes Torneo tenía los partidos directamente). Torneo además valida la programación de partidos (estadio, clubes, árbitros y período), carga las nóminas rechazando a los suspendidos y registra las sanciones.
Sanciones	Sancion representa una fecha de suspensión y se marca como cumplida (no hace falta fechasSuspension). La crea Torneo y se asocia con el jugador, el club y el partido de origen.

Inscripción	El plantel (18 a 30 jugadores) se valida completo al crear la Inscripcion, porque se inscribe "con el plantel ya armado". Por eso no tiene agregarJugador, quitarJugador ni validarPlantel.
Árbitros	Se agregó la asociación Arbitro–Club (clubes de los que es hincha) para validar la designación de árbitros (RN19).
Operación de partidos	La coordinación de inicio, suspensión, eventos, anulaciones y finalización se integra mediante ControladorPartido, que también coordina el envío de notificaciones cuando corresponde. Partido valida cada evento, y los eventos responden por polimorfismo (golesPara, calcularXG, expulsa…), sin usar instanceof.
Reportes	Se agregaron FilaTabla y Goleador (implementan Comparable) y ComparadorTabla (implementa Comparator) para ordenar la tabla y los goleadores con los desempates de la RN25. obtenerGoleadores devuelve una lista de Goleador.
Administración CU01	Se añadieron opciones de modificación y baja para las seis categorías principales. La modificación cubre campos seleccionados por categoría, no necesariamente todos los atributos de cada entidad, y cada dato nuevo se valida. La baja solo se permite si el dato no está en uso.

Supuestos agregados durante la codificación:

- Las dos amarillas de una expulsión por doble amarilla no suman para el límite de 5: solo cuenta la roja (1 fecha), para no castigar dos veces el mismo hecho.
- En la proyección por xG, cada partido que falta se simula con 10 jugadas; en cada una el club convierte con probabilidad xG/10. Un club que todavía no jugó tiene xG promedio 0.
- La notificación del final del partido se envía con el minuto 90.
- El plantel que se inscribe en un torneo son los jugadores con contrato vigente en el club.

2. Dificultades encontradas

- **Integración de trabajo paralelo:** al trabajar en ramas distintas fue necesario sincronizar cambios y resolver diferencias en archivos compartidos, especialmente en la interfaz y la integración de controladores. Al principio, además, dos integrantes programaron Torneo e Inscripcion al mismo tiempo, lo que generó conflictos en Git y obligó a unir las dos versiones. A partir de ahí avisamos en el grupo qué clase tomaba cada uno antes de empezar.
- **Uso de Git y GitHub en equipo:** ramas, pull requests, revisiones y resolución de conflictos fueron nuevos para varios integrantes. Por ejemplo, aprendimos que hacer push no alcanza: hace falta abrir el pull request para que el trabajo se revise y entre a main.
- **Configuración del entorno:** tuvimos problemas con IntelliJ (módulo desconfigurado, versión de Java distinta) y hubo que agregar JUnit para ejecutar las pruebas.

- **Persistencia de objetos:** fue necesario considerar la serialización de todo el grafo de objetos, el tipo esperado al recuperar datos y los errores posibles de lectura y escritura (archivo inexistente, vacío o dañado).
- **Reglas de negocio encadenadas:** una modificación o baja no depende únicamente del objeto seleccionado; puede afectar inscripciones, contratos, partidos, nóminas, patrocinios y estadísticas históricas.
- **Reglas ambiguas:** algunas situaciones que la consigna no define nos obligaron a decidir y documentar un supuesto (la doble amarilla, el desempate de la tabla o la simulación con xG).
- **Requisitos que se nos pasaron:** cerca del final detectamos que el CU01 pedía modificación y baja, y el menú solo tenía alta y consulta. Lo completamos antes de la entrega.
- **Código que no podíamos explicar:** en el menú y en las pruebas habíamos usado estructuras que no vimos en la materia (lambdas y assertThrows). Como tenemos que poder defender todo el código, las reemplazamos por switch clásico con break y por try/catch con fail().
- **Pruebas reproducibles:** las pruebas unitarias necesitan JUnit 5. Documentamos en el README cómo agregarlo y ejecutarlas desde IntelliJ; con esa configuración pasan las 174 pruebas.
- **Ajuste del alcance:** algunas funciones de administración, como la modificación de datos, quedaron implementadas sobre atributos concretos de cada categoría y deben explicarse con ese alcance real.
- **Mantener el diagrama al día:** mientras programábamos aparecieron clases y métodos nuevos. Al final actualizamos el diagrama completo y registramos cada cambio con su justificación.

3. Distribución de tareas

Trabajamos con una rama por funcionalidad y pull requests revisados por otro integrante; todo queda registrado en el historial del repositorio.

Integrante	Aportes principales
Fausto Fernandez	Creación y administración del repositorio, revisión e integración de los pull requests. Primera versión del documento de la Etapa 1 y diagramas de secuencia. Implementación de la persistencia (RepositorioArchivo y PersistenciaException), carga y guardado de Datos, conexión de los controladores con el menú y pruebas del componente de persistencia.
Ignacio Mendes	Diagramas de clases de la Etapa 1 y su actualización para la Etapa 2, con el resumen de cambios. Implementación de Partido, los eventos y sus validaciones, las sanciones por tarjetas, las estadísticas de jugadores, el fixture, las nóminas y las notificaciones. Menú de consola y pruebas unitarias de Partido, Torneo, notificaciones y CU01.
Luka Kishima	Casos de uso y justificación de SOLID y GRASP en el documento de la Etapa 1. Implementación de las clases base (personas, clubes, estadios y contratos), Torneo e inscripciones, sponsors y patrocinios, reportes y proyecciones, controladores, y modificación y baja de datos (CU01). Pruebas de reportes y patrocinios, README y plan de pruebas funcionales.

4. Reflexión del grupo

El desarrollo permitió pasar de un modelo conceptual a una aplicación con módulos que interactúan entre sí. El uso de Git y GitHub facilitó dividir el trabajo, aunque al principio nos dio problemas, mantener ramas por funcionalidad y revisar la integración mediante pull requests.

La implementación también mostró la importancia de mantener las reglas de negocio en el dominio y no resolverlas únicamente desde el menú.

Como aprendizaje, el grupo identificó que integrar no consiste solamente en unir código: también es necesario comprobar que las relaciones entre objetos sigan siendo válidas, que la persistencia conserve la información y que las pruebas puedan ejecutarse en un entorno bueno.